

Porównanie efektywności algorytmów dokładnych i heurystycznych dla Problemu Komiwożera (TSP)

Kacper Piekut

Numer albumu: 280879

Wydział: W4N, Rok akademicki: 2025/2026, Semestr: 6

Politechnika Wrocławska

Streszczenie—W niniejszym sprawozdaniu przedstawiono analizę i porównanie czterech algorytmów rozwiązujących Problem Komiwożera w wariacie symetrycznym (STSP) oraz asymetrycznym (ATSP): Przeglądu Zupełnego (Brute-Force), błędzenia losowego (RAND), Najbliższego Sąsiada (NN) oraz powtarzalnego Najbliższego Sąsiada (RNN). Badania przeprowadzono na wygenerowanych instancjach losowych oraz na grafach z biblioteki TSPLIB, w tym na specyficznych instancjach technologicznych (VLSI/PCB). Wyniki potwierdzają wykładniczą złożoność algorytmu Brute-Force oraz ukazują charakterystykę zbieżności algorytmu losowego. Wykazano również drastyczny kompromis pomiędzy czasem obliczeń a gwarancją optymalności.

Index Terms—Problem Komiwożera, Algorytmy Heurystyczne, Złożoność Obliczeniowa, Cykl Hamiltona, TSPLIB, VLSI, TSP, ATSP.

I. WSTĘP

A. Hipotezy badawcze

Przed przystąpieniem do pomiarów sformułowano następujące hipotezy, odpowiadające na pytania badawcze:

- **H1 (Złożoność Brute-Force):** Czas obliczeń algorytmu dokładnego rośnie w tempie silni ($O((N - 1)!)$), co sprawi, że przekroczy on założony limit 30 minut już dla instancji $N \geq 15$, niezależnie od tego, czy graf jest symetryczny, czy asymetryczny.
- **H2 (Kompromis i wyznaczanie UB):** Istnieje drastyczny kompromis między czasem a jakością. Metody heurystyczne wyznaczą rozwiązania o kilka rzędów wielkości szybciej niż metoda dokładna, co czyni je bardzo efektywnymi metodami do błyskawicznego wyznaczania górnego ograniczenia (UB) dla algorytmów dokładnych.
- **H3 (Zbieżność RAND i warunek stopu):** Prawdopodobieństwo znalezienia lepszej trasy przez algorytm losowy maleje z upływem czasu, prowadząc do stagnacji. Pozwala to na zastąpienie sztywnego limitu czasowego warunkiem braku poprawy, drastycznie skracając czas badań bez utraty wiarygodności wyników.
- **H4 (Wpływ wierzchołka startowego):** Wynik klasycznego algorytmu zachłannego (NN) jest krytycznie uzależniony od wyboru początkowego miasta. Iteracyjne sprawdzenie każdego punktu startowego (RNN) zagwarantuje znalezienie trasy o koszcie mniejszym lub równym.

II. METODOLOGIA BADAŃ

A. Środowisko testowe i dane wejściowe

Badania przeprowadzono na laptopie ASUS TUF Gaming F15 (procesor Intel Core i5-11400H 2.7-4.5 GHz, 16 GB RAM DDR4, dysk SSD NVMe 512 GB) pod kontrolą systemu Windows 11 Home (64-bit). Środowiskiem programistycznym był język C++ (kompilator MinGW GCC). W badaniach wykorzystano dwa źródła danych:

- **Generowane losowo grafy pełne:** wygenerowano zestawy instancji symetrycznych i asymetrycznych o rozmiarach $N \in \langle 6, 15 \rangle$ dla algorytmu Brute-Force oraz $N = 11$ do weryfikacji kompromisu czasowego. Krawędzie losowano z rozkładu jednostajnego z przedziału $\langle 1, 100 \rangle$.
- **Biblioteka TSPLIB:** wykorzystano klasyczne instancje geograficzne oraz specjalistyczne instancje technologiczne VLSI.

B. Opis zaimplementowanych algorytmów

Badaniom poddano cztery implementacje algorytmów rozwiązujących Problem Komiwożera:

- **Błądzenie Losowe (RAND):** Metoda losująca permutacje w pętli aż do momentu osiągnięcia limitu czasu lub odnalezienia rozwiązania optymalnego. Zastosowano mechanizm blokowania pierwszego miasta w celu zminimalizowania ryzyka powtórnej wylosowania tego samego cyklu Hamiltona.
- **Najbliższy Sąsiad (NN):** Metoda zachłanna, która zaczynając od zadanego wierzchołka, w każdym kroku dobiera najbliższe, nieodwiedzone jeszcze miasto.
- **Powtarzalny Najbliższy Sąsiad (RNN):** Rozszerzenie algorytmu NN. Metoda iteruje po każdym z N wierzchołków grafu traktując go jako potencjalny start. Dodatkowo, implementacja uwzględnia obsługę remisów (równych wag) poprzez tworzenie rozgałęzień i sprawdzanie wszystkich możliwych dróg o tym samym lokalnym koszcie minimalnym.
- **Przegląd Zupełny (Brute-Force):** Metoda dokładna generująca wszystkie możliwe permutacje wierzchołków. W celu optymalizacji przestrzeni poszukiwań również zastosowano blokowanie pierwszego wierzchołka (jak w metodzie RAND), redukując liczbę permutacji z $N!$ do $(N - 1)!$.

C. Konfiguracja eksperymentu

Aplikacja testowa została zaprojektowana w sposób wykluczający twarde kodowanie. Całym przepływem steruje zewnętrzny plik tekstowy `config.txt`. Przykładowa konfiguracja uruchamiająca badanie algorytmu RNN z limitem 15 minut (900 sekund) i 100 powtórzeniami prezentuje się następująco:

```
ALGORITHM = RNN
GRAPH_PATH = data/tsplib/tsp/berlin52.tsp
GENERATE_NEW = false
OPTIMAL_COST = 7542
TIME_LIMIT_SECONDS = 900
REPETITIONS = 100
SAVE_TO_CSV = true
```

D. Procedura badawcza i miary oceny

Eksperymenty podzielono na dwie fazy. W pierwszej fazie weryfikowano algorytmy dokładne, mierząc całkowity czas znalezienia globalnego optimum w zależności od rozmiaru N oraz typu symetrii grafu. W drugiej fazie weryfikowano metody heurystyczne. Jakość rozwiązań heurystycznych oceniano za pomocą błędu względnego δ , który wyraża procentowy wzrost kosztu znalezionej trasy względem optymalnego, znanego rozwiązania:

$$\delta = \frac{Koszt_{alg} - Koszt_{opt}}{Koszt_{opt}} \cdot 100\% \quad (1)$$

Aby zniwelować wpływ procesów systemowych na precyzję pomiarów czasu, procedura badawcza zakładała wielokrotne uruchamianie algorytmu dla danej instancji (zgodnie z parametrem `REPETITIONS`). Program dokonywał agregacji wyników, obliczając średni czas wykonania, podczas gdy koszt końcowy stanowił najlepszy (minimalny) wynik uzyskany ze wszystkich powtórzeń.

E. Teoretyczna złożoność i przestrzeń poszukiwań

Dla algorytmu dokładnego (Brute-Force) oraz algorytmu losowego (RAND) wspólną przestrzenią poszukiwań jest zbiór wszystkich cykli Hamiltona. Dzięki optymalizacji zablokowania startu, rozmiar przestrzeni P wynosi:

$$P = (N - 1)! \quad (2)$$

Z kolei dla algorytmów zachłannych zdefiniowano teoretyczną złożoność obliczeniową. Algorytm Najbliższego Sąsiada (NN) w każdym z N kroków przeszukuje listę malejącej liczby nieodwiedzonych sąsiadów, co daje złożoność rzędu $O(N^2)$. Algorytm powtarzalny (RNN), sprawdzając dodatkowo wszystkie starty, zwiększa tę złożoność do $O(N^3)$.

F. Metodyka szacowania maksymalnego rozmiaru instancji

Zakładając, że czas wykonania algorytmu t jest proporcjonalny do jego teoretycznej złożoności $f(N)$, można wprowadzić sprzętową stałą proporcjonalności c :

$$t \approx c \cdot f(N) \quad (3)$$

Znając czas wykonania t dla konkretnego, zbadanego rozmiaru instancji N , stałą c dla danego środowiska obliczeniowego wyznacza się ze wzoru:

$$c = \frac{t}{f(N)} \quad (4)$$

Posiadając wyznaczoną stałą proporcjonalności, można przekształcić równanie, aby oszacować maksymalny rozmiar instancji N_{max} , który system jest w stanie przetworzyć w limicie czasowym T_{limit} . Dla algorytmów o złożoności kwadratowej $O(N^2)$ oraz sześcienniej $O(N^3)$, wzory przyjmują odpowiednio postać:

$$N_{max} = \sqrt{\frac{T_{limit}}{c}} \quad (5)$$

$$N_{max} = \sqrt[3]{\frac{T_{limit}}{c}} \quad (6)$$

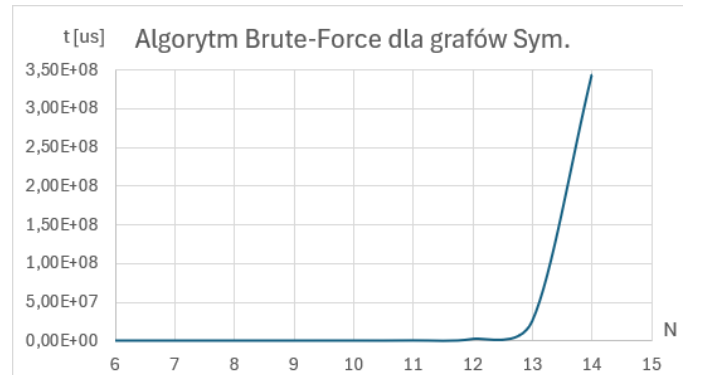
III. WYNIKI BADAŃ

A. Złożoność algorytmu Brute-Force (Sym. vs Asym.)

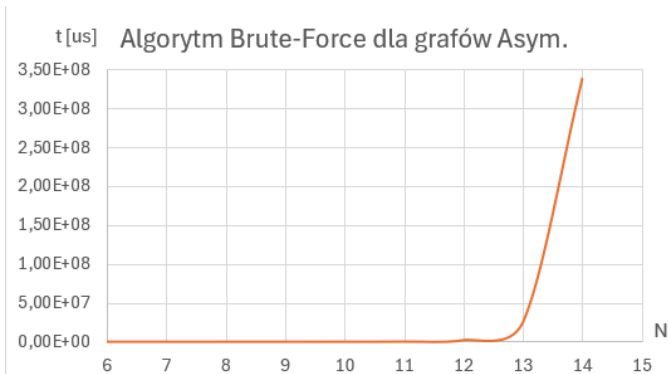
Wygenerowano dwa zbiory grafów pełnych o rozmiarach od $N = 6$ do $N = 15$. Zestawienie czasów wykonania algorytmu Brute-Force przedstawia Tabela I oraz wykresy 1 i 2.

Tabela I
PORÓWNANIE CZASU WYKONANIA ALGORYTMU BRUTE-FORCE DLA GRAFÓW SYMETRYCZNYCH I ASYMETRYCZNYCH

N	P	Czas - Sym. [us]	Czas - Asym. [us]
6	120	$9,60 \cdot 10^0$	$1,24 \cdot 10^1$
7	720	$4,26 \cdot 10^1$	$3,96 \cdot 10^1$
8	5 040	$2,86 \cdot 10^2$	$2,64 \cdot 10^2$
9	40 320	$2,17 \cdot 10^3$	$2,14 \cdot 10^3$
10	362 880	$1,99 \cdot 10^4$	$1,96 \cdot 10^4$
11	3 628 800	$1,85 \cdot 10^5$	$1,87 \cdot 10^5$
12	39 916 800	$2,11 \cdot 10^6$	$2,12 \cdot 10^6$
13	479 001 600	$2,52 \cdot 10^7$	$2,53 \cdot 10^7$
14	6 227 020 800	$3,43 \cdot 10^8$	$3,39 \cdot 10^8$
15	87 178 291 200	TimeOut	TimeOut



Rysunek 1. Zależność czasu wykonywania od rozmiaru instancji (N) dla algorytmu Brute-Force (grafy symetryczne).



Rysunek 2. Zależność czasu wykonywania od rozmiaru instancji (N) dla algorytmu Brute-Force (grafy asymetryczne).

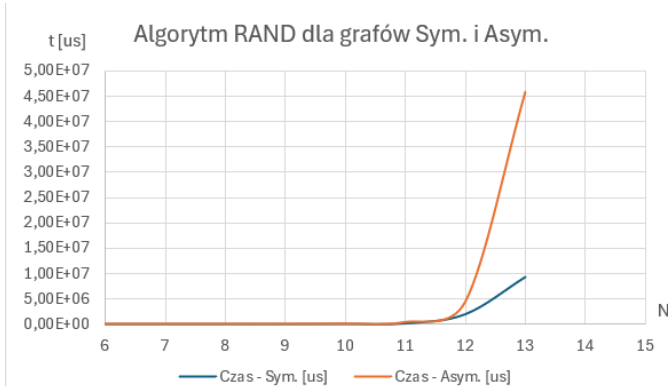
B. Złożoność i ograniczenia czasowe algorytmu RAND

Przeprowadzono badanie algorytmu błędzenia losowego (RAND) na wygenerowanych instancjach o rozmiarach od $N = 6$ do $N = 13$ (Tabela II, wykres 3). Ponieważ założony limit czasowy (30 minut) był wystarczający na odnalezienie optimum, błąd względny dla wszystkich poniższych pomiarów wyniósł 0%. W związku z tym w poniższym zestawieniu pominięto kolumnę błędu, skupiając się wyłącznie na drastycznym wzroście czasu wykonania algorytmu.

Następnie, oszacowano maksymalny rozmiar instancji N ,

Tabela II
CZAS POSZUKIWANIA ROZWIĄZANIA OPTYMALNEGO PRZEZ ALGORYTM RAND

N	P	Czas - Sym. [us]	Czas - Asym. [us]
6	120	$7,60 \cdot 10^0$	$2,10 \cdot 10^1$
7	720	$4,96 \cdot 10^1$	$2,80 \cdot 10^1$
8	5 040	$1,62 \cdot 10^2$	$2,28 \cdot 10^2$
9	40 320	$5,94 \cdot 10^2$	$3,82 \cdot 10^3$
10	362 880	$1,96 \cdot 10^4$	$4,72 \cdot 10^4$
11	3 628 800	$1,32 \cdot 10^5$	$3,51 \cdot 10^5$
12	39 916 800	$1,95 \cdot 10^6$	$4,38 \cdot 10^6$
13	479 001 600	$9,32 \cdot 10^6$	$4,60 \cdot 10^7$



Rysunek 3. Zależność czasu wykonywania od rozmiaru instancji (N) dla algorytmu RAND.

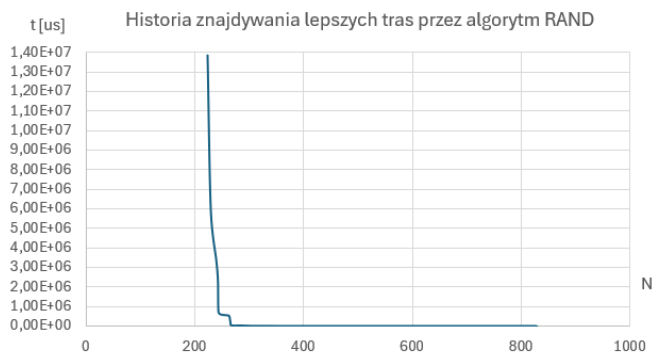
dla której algorytm losowy jest w stanie znaleźć optimum w

rozsądnym czasie 30 minut. Należy zaznaczyć, że przestrzeń poszukiwań algorytmu RAND to zawsze zbiór wszystkich cykli Hamiltona wynoszący $P = (N - 1)!$, niezależnie od tego, czy instancja jest generowana losowo, pochodzi z TSPLIB czy jest siatką VLSI (algorytm losuje trasy w ciemno, ignorując wagi krawędzi). Z tego powodu oszacowanie maksymalnego N opiera się wyłącznie na uniwersalnej przepustowości obliczeniowej zaimplementowanego algorytmu. W celu weryfikacji przepustowości uruchomiono algorytm na instancji berlin52.tsp z rygorystycznym limitem 1 minuty. W tym czasie program wygenerował i ocenił dokładnie 304 686 769 tras. Dokonując liniowej ekstrapolacji, w czasie 30 minut algorytm jest w stanie sprawdzić C_{max} unikalnych tras:

$$C_{max} = 30 \cdot 304\,686\,769 \approx 9,14 \cdot 10^9 \text{ tras} \quad (7)$$

C. Historia zbieżności algorytmu RAND

W celu szczegółowego zbadania charakterystyki pracy algorytmu losowego, zarejestrowano proces poprawy najlepszego znalezionej trasy w trakcie trwania obliczeń. Analizę przeprowadzono na podstawie pliku z historią wykonania (history RAND.csv) dla instancji o rozmiarze $N = 13$. Zarejestrowane punkty pomiarowe, obrazujące zmianę kosztu najkrótszej odnalezionej trasy w funkcji upływającego czasu, naniesiono na wykres przedstawiony na Rysunku 4.



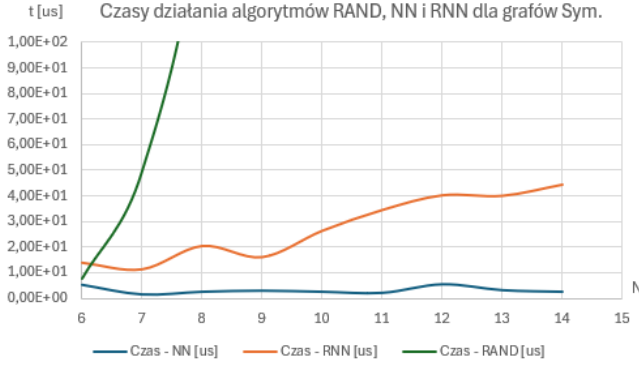
Rysunek 4. Krzywa zbieżności algorytmu RAND dla instancji o rozmiarze $N = 13$. Wykres przedstawia zmianę kosztu najlepszego znalezionej trasy w funkcji czasu wyrażonego w mikrosekundach.

D. Efektywność i błąd względny algorytmów NN i RNN

Przeprowadzona została analiza metod: algorytmu Najbliższego Sąsiada (NN) oraz jego rozszerzonej wersji, sprawdzającej każdy możliwy wierzchołek startowy (RNN). W pierwszej kolejności zestawiono czasy wykonania tych metod z algorytmem losowym (RAND). Zależność tę, w skali logarymicznej, przedstawiono na Rysunku 5. Wyniki dla wygenerowanych instancji symetrycznych i asymetrycznych zestawiono w Tabeli III oraz zobrazowano na Rysunku 6.

E. Maksymalny rozmiar instancji dla algorytmów NN i RNN dla limitu czasowego

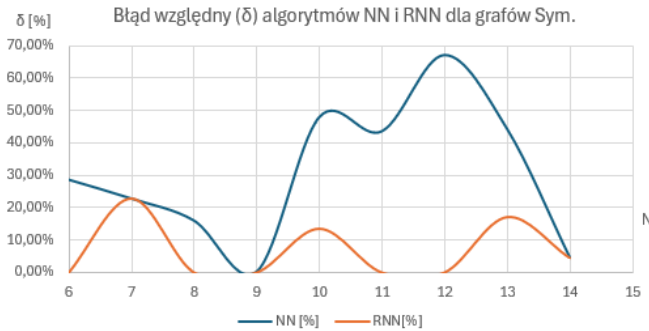
Podjęta została próba oszacowania maksymalnego rozmiaru grafu (N), dla którego algorytmy zachłanne znajdą rozwiąza-



Rysunek 5. Porównanie czasu wykonania algorytmów RAND, NN i RNN w zależności od rozmiaru instancji (N). Oś Y w skali logarytmicznej.

Tabela III
PORÓWNANIE BŁĘDU WZGLĘDNEGO ALGORYTMÓW NN I RNN

N	Instancja	δ NN [%]	δ RNN [%]
6	sym_6	28,47%	0,00%
7	sym_7	22,63%	22,63%
8	sym_8	15,81%	0,00%
9	sym_9	0,00%	0,00%
10	sym_10	47,85%	13,40%
11	sym_11	43,48%	0,00%
12	sym_12	67,10%	0,00%
14	sym_14	4,44%	4,44%
6	asym_6	0,00%	0,00%
7	asym_7	29,73%	0,54%
8	asym_8	59,12%	2,19%
9	asym_9	56,35%	0,00%
10	asym_10	66,03%	3,21%
11	asym_11	62,31%	6,92%
12	asym_12	89,08%	38,66%
14	asym_14	40,98%	39,51%



Rysunek 6. Wykres słupkowy prezentujący zestawienie błędu względnego dla algorytmów NN i RNN na badanych instancjach.

nie w czasie 15 minut ($9 \cdot 10^8 \mu s$). Ponieważ zbadane metody charakteryzują się złożonością wielomianową, wyliczeń dokonano na podstawie stałej proporcjonalności c wyznaczonej dla instancji $N = 14$. Dla algorytmu Najbliższego Sąsiada (NN) o złożoności $\mathcal{O}(N^2)$ czas wykonania dla $N = 14$ wyniósł $t = 2,6 \mu s$.

$$c_{NN} = \frac{t}{N^2} = \frac{2,6}{14^2} \approx 0,0132 \mu s \quad (8)$$

Przyrównując to do limitu 15 minut, otrzymujemy maksymalny rozmiar problemu:

$$N_{max_NN} = \sqrt{\frac{9 \cdot 10^8}{0,0132}} \approx 261\,000 \text{ wierzchołków} \quad (9)$$

Dla algorytmu powtarzalnego (RNN) o złożoności $\mathcal{O}(N^3)$ czas dla $N = 14$ wyniósł $t = 44,6 \mu s$.

$$c_{RNN} = \frac{t}{N^3} = \frac{44,6}{14^3} \approx 0,0162 \mu s \quad (10)$$

$$N_{max_RNN} = \sqrt[3]{\frac{9 \cdot 10^8}{0,0162}} \approx 3\,800 \text{ wierzchołków} \quad (11)$$

F. Weryfikacja błędu względnego na instancjach TSPLIB i VLSI

Została przeprowadzona konfrontacja zaimplementowanych algorytmów z rzeczywistymi, ustandaryzowanymi instancjami problemu z biblioteki TSPLIB (berlin52.tsp, ft53.atsp) oraz siatką punktów VLSI (pia3056.tsp). Algorytmy zachłanne (NN i RNN) z racji swojej wielomianowej złożoności rozwiązały te instancje błyskawicznie. Algorytmowi losowemu (RAND) narzucono w tym teście rygorystyczny limit czasowy wynoszący 1 minutę, badając jakość rozwiązania, jakie jest w stanie wygenerować w krótkim oknie czasowym.

Tabela IV
BŁĄD WZGLĘDNY ALGORYTMÓW DLA INSTANCIJ TSPLIB I VLSI

Instancja	Typ	N	δ NN	δ RNN	δ RAND (60s)
berlin52	STSP	52	19,07%	8,47%	161,75%
ft53	ATSP	53	25,75%	9,63%	434,63%
pia3056	VLSI	3056	22,03%	16,82%	2186,41%

IV. DYSKUSJA I WNIOSKI

A. Weryfikacja hipotez badawczych

Na podstawie przeprowadzonych eksperymentów i zebranych danych analitycznych, zweryfikowano hipotezy postawione we wstępie:

- **H1 (Złożoność Brute-Force) - Potwierdzona.** Pomiary czasu wykazały wzrost zgodny z krzywą silni. Dla instancji $N = 15$ algorytm nie odnalazł rozwiązania w racjonalnym czasie i przekroczył narzucony limit, niezależnie od tego, czy badany graf był symetryczny, czy asymetryczny.
- **H2 (Kompromis i wyznaczanie UB) - Potwierdzona.** Zaobserwowano przepaść wydajnościową. Algorytmy

heurystyczne rozwiązują problemy wielokrotnie szybciej, dostarczając bardzo dobre wyniki startowe. Czyni to z nich perfekcyjne narzędzie do błyskawicznego wyznaczenia Górnego Ograniczenia (UB).

- **H3 (Zbieżność RAND i warunek stopu) - Potwierdzona.** Analiza krzywej zbieżności uwidoczniła szybkie wejście algorytmu losowego w stan długotrwałej stagnacji. Zastąpienie limitu czasowego warunkiem ucinania algorytmu po z góry ustalonej liczbie pustych iteracji pozwala drastycznie zredukować czas poszukiwań przy zachowaniu identycznej jakości.
- **H4 (Wpływ wierzchołka startowego) - Potwierdzona.** Narzucenie pojedynczego miasta startowego w algorytmie NN prowadziło w wielu przypadkach do suboptymalnych decyzji. Zastosowanie iteracji po wszystkich wierzchołkach (RNN) całkowicie zniwelowało to ryzyko, co przełożyło się na bezwzględną poprawę precyzji.

B. Złożoność obliczeniowa i skalowalność

Wyznaczona została złożoność obliczeniową badanych metod. Casy wykonania algorytmu Brute-Force odwzorowują krzywą silni, potwierdzając jego teoretyczną złożoność $O((N-1)!)$. Dodatkowo, brak widocznych różnic w czasie dla grafów symetrycznych i asymetrycznych dowodzi, że czas działania algorytmu jest całkowicie niezależny od topologii i wagi krawędzi. W drastycznym kontraście pozostają algorytmy zachłanne. Dla reprezentacji macierzowej, algorytm Najbliższego Sąsiada (NN) w każdym z N kroków przeszukuje listę malejącej liczby nieodwiedzonych wierzchołków. Stanowi to sumę ciągu arytmetycznego, dając złożoność wielomianową $O(N^2)$. Z kolei wersja powtarzalna (RNN), jako rozszerzenie polegające na iteracji przez wszystkie N możliwych miast startowych, powiększa ten czas N -krotnie, podnosząc złożoność do $O(N^3)$. Jak wykazały badania (Rys. 5), oba te rzędy wielkości dla grafów z rzędu $N \leq 15$ dają czasy egzekucji liczone w ułamkach milisekund, gwarantując potężną skalowalność niedostępną algorytmom dokładnym. Narzucanie limitów czasowych (np. 30 minut) jest koniecznością dla algorytmów klasy Brute-Force oraz RAND z uwagi na fizyczną i technologiczną niemożliwość przeszukania przestrzeni rosnącej w tempie silni. Z kolei dla algorytmów wielomianowych (NN i RNN) limit 15 minut jest przydatny - pozwala on na przetwarzanie gigantycznych grafów, przez co w praktyce dla mniejszych instancji limit ten nie ma żadnego znaczenia i nigdy nie zostaje osiągnięty.

C. Zjawisko stagnacji i ograniczenie czasu w algorytmie RAND

Na podstawie matematycznych obliczeń przepustowości wykazano, że 30-minutowe ramy czasowe są niewystarczające, aby algorytm RAND mógł odnaleźć optimum dla grafów z biblioteki TSPLIB. Przeanalizowano również możliwość zastąpienia sztywnego, 30-minutowego limitu innym, bardziej racjonalnym kryterium stopu. Analiza wygenerowanej krzywej zbieżności (Rys. 4) daje jednoznaczną odpowiedź: **czas działania algorytmu probabilistycznego można znacząco**

skrócić poprzez zastosowanie warunku braku poprawy rozwiązania (stagnacji). Wykres udowadnia, że metoda losowa początkowo błyskawicznie znajduje akceptowalne trasy, po czym wpada w bardzo długie, płaskie fazy beczynności. Znacznie efektywniejszym i wiarygodnym kryterium stopu jest odliczanie pustych iteracji - jeżeli przez ustalony próg (np. 1 milion wylosowanych tras) algorytm nie znajdzie lepszego kosztu, dalsze obliczenia powinny zostać przerwane. Aby zapewnić brak powtarzających się permutacji przy zachowaniu w pełni losowego charakteru, jedną z najskuteczniejszych metod jest zastosowanie generatora liczb pseudolosowych (Mersenne Twister `std::mt19937` ze standardu C++ w połączeniu ze `std::shuffle`), co gwarantuje równomierny rozkład prawdopodobieństwa tasowania wierzchołków.

D. Efektywność heurystyk i wyznaczanie ograniczeń (UB)

Odpowiadając na jedno z kluczowych pytań projektu: **kiedy i dlaczego rozwiązanie RNN jest korzystniejsze niż NN?** Badania (Tab. III) udowodniły, że klasyczny algorytm NN, ze względu na swój skrajny optymizm lokalny, podejmuje gorsze decyzje. Rozwiązanie RNN jest bezwzględnie korzystniejsze w każdej sytuacji, w której nie jesteśmy rygorystycznie ograniczeni czasem. Poświęcając minimalnie więcej mocy obliczeniowej (z $O(N^2)$ na $O(N^3)$), omijamy ryzyko wyboru najgorszego wierzchołka startowego, z którego nie ma dobrego wyjścia do reszty grafu, często niwelując błąd względny do 0% na małych instancjach. **Górne i Dolne Ograniczenie (UB i LB):** W kontekście rozwiązywania TSP, Górne Ograniczenie (UB) to koszt najgorszego akceptowalnego rozwiązania, stanowiący próg odcinania gorszych tras. Dolne Ograniczenie (LB) to z kolei teoretyczny koszt idealnej trasy. Jak wykazały pomiary, algorytmy NN i RNN są perfekcyjnymi, superszybкими generatorami bardzo silnego (niskiego) UB. W ułamek sekundy dostarczają trasę o dobrej jakości. Z kolei używanie do tego celu algorytmu RAND jest nieefektywne z powodu osiągania gorszych wyników czasowych i jakościowych.

E. Wpływ parametrów instancji na złożoność zbadanych algorytmów

Odpowiadając na pytania badawcze, przeanalizowano wpływ struktury grafu na działanie algorytmów:

- **Gęstość grafu:** Zbadane tu algorytmy (Brute-Force, RAND, NN, RNN) w swojej naiwnej postaci zakładają pełną macierz sąsiedztwa i nie korzystają z odcinania. W związku z tym brak krawędzi (mniejsza gęstość) nie zmniejsza liczby iteracji - program nadal musi sprawdzić $O((N-1)!)$ lub wykonać N^k kroków.
- **Wiele ścieżek o optymalnym koszcie:** Fakt ten nie ma absolutnie żadnego wpływu na algorytmy Brute-Force, NN czy RNN, które działają deterministycznie lub muszą sprawdzić całą przestrzeń. Ma jednak pozytywny wpływ na algorytm RAND - zagęszczenie optymalnych rozwiązań w przestrzeni drastycznie zwiększa prawdopodobieństwo trafienia optimum w krótszym czasie.

- **Zakres wag krawędzi:** Ani duża, ani mała wariancja wag nie wpływa na złożoność zbadanych metod. Będą one weryfikować tę samą liczbę kombinacji niezależnie od wielkości liczb.

F. Podsumowanie ogólne

Przeprowadzone eksperymenty uwydatniły wymaganie kompromisu pomiędzy czasem a optymalnością w analizie problemu komiwojażera. Algorytm Przeglądu Zupełnego (Brute-Force), choć gwarantuje znalezienie 100% optimum, posiada zerową przydatność komercyjną ze względu na wykładniczą barierę rozwiązań czasowych. Z kolei przebadane algorytmy konstrukcyjne, na czele z wielokrotnym wariantem Najbliższego Sąsiada (RNN), stanowią doskonały punkt wyjścia dla dalszych badań. Skalowalność wielomianowa przy jednoczesnym generowaniu trasy o akceptowalnym błędzie względnym czyni z nich idealnych dostawców rozwiązań startowych (UB) dla o wiele bardziej zaawansowanych grafów.

LITERATURA

- [1] *VLSI TSP Data*, University of Waterloo. [Online]. Dostępne: <https://www.math.uwaterloo.ca/tsp/vlsi/index.html>
- [2] W. Cook et al., *The Traveling Salesman Problem*, University of Waterloo. [Online]. Dostępne: <https://www.math.uwaterloo.ca/tsp/index.html>
- [3] K. Kwieciński, *Lekcja 5: Droga i cykl Hamiltona*, eTrapez. [Online]. Dostępne: <https://online.etrpez.pl/lesson/lekcja-5-droga-i-cykl-hamiltona/>
- [4] *Asymptotyczne tempo wzrostu*, Wikipedia, wolna encyklopedia. [Online]. Dostępne: https://pl.wikipedia.org/wiki/Asymptotyczne_tempo_wzrostu
- [5] *std::shuffle vs std::random_shuffle in C++*, GeeksforGeeks. [Online]. Dostępne: https://www.geeksforgeeks.org/cpp/shuffle-vs-random_shuffle-c/
- [6] *std::next_permutation and prev_permutation in C++*, GeeksforGeeks. [Online]. Dostępne: https://www.geeksforgeeks.org/cpp/stdnext_permutation-prev_permutation-c/
- [7] *std::mt19937 class in C++*, GeeksforGeeks. [Online]. Dostępne: <https://www.geeksforgeeks.org/cpp/stdmt19937-class-in-cpp/>